CS 61A — Week 1

Expressions, names, functions, environment diagrams, print and None (Lectures 1–2, Composing Programs 1.1–1.3)

§1 This week

§1.1 Expressions and how Python evaluates them

The question. A program is text. What exactly does Python do, step by step, to turn the text `mul(add(4, mul(4, 6)), add(3, 5))` into the number `224`? Every "What Would Python Display" exam question is this question.

- An **expression** describes a computation and **evaluates to a value**. `2026`, `'hello'`, `2 + 3`, `max(1, 2)` are expressions.
- Values have types. This week: numbers (`6`, `2.5`), strings (`'are you human'`), functions (`pow`), and the special value `None`.
- A **call expression** has the form *operator*`(`*operand*`, ...)`. In `max(1 + 1, 3)` the operator is `max` and the operands are `1 + 1` and `3`. The operator and the operands are themselves expressions.
- Every arithmetic expression can be written as a call: `2 + 3` is `add(2, 3)`, `2 * 3` is `mul(2, 3)` (after `from operator import add, mul`).

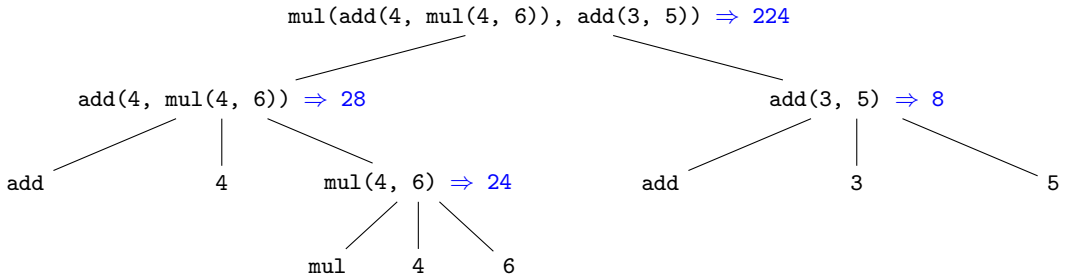
Evaluation procedure for a call expression.

1. Evaluate the operator. Its value must be a function.
2. Evaluate each operand, from left to right. Their values are the **arguments**.
3. Apply the function to the arguments. The result is the value of the call expression.

The idea

The procedure refers to itself: step 2 says "evaluate each operand", and an operand may be another call expression, which is evaluated by the same three steps. So a nested expression is evaluated from the inside out, and among operands at the same level, from left to right.

The order matters as soon as evaluating an operand *does* something visible, such as printing. On exams, the order of the displayed lines is decided by this procedure and by nothing else.



Expression tree. Each call expression is evaluated after its operator and operands. Order in which the calls finish: `mul(4, 6)`, then `add(4, 24)`, then `add(3, 5)`, then `mul(28, 8)`.

Arithmetic operators you will use constantly.

- `/` is true division and always gives a float: `6 / 3` is `2.0`. `//` is floor division: `7 // 2` is `3`. `%` is remainder: `7 % 2` is `1`. `**` is power: `2 ** 10` is `1024`.
- For a positive integer `n`: `n % 10` is its last digit and `n // 10` is `n` with the last digit removed. (`2026 % 10` is `6`; `2026 // 10` is `202`.)
- `n % d == 0` tests whether `d` divides `n` evenly.

§1.2 Names and assignment

The question. After `x = 2`, then `y = x + 1`, then `x = 5`: what is `y`?

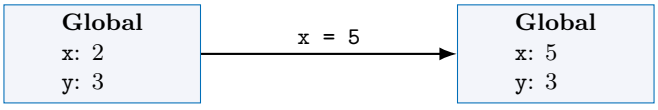
Execution rule for an assignment statement `name = expression`:

1. Evaluate the expression on the right.
2. Bind the name on the left to that *value* in the current frame (replacing any earlier binding of the name).

The idea

A name is bound to a value, never to another name or to an expression. When `y = x + 1` runs, Python computes `3` and binds `y` to `3`. Nothing remembers that the `3` came from `x`. So changing `x` later cannot change `y`.

```
>>> x = 2
>>> y = x + 1
>>> x = 5
>>> y
3
```



The global frame before and after `x = 5`. Rebinding `x` changes only the binding of `x`; `y` keeps the value 3 it was given, because a name is bound to a value, not to the expression that produced it.

- **Functions are values, so names can be bound to them.** From lecture:

```
>>> max = pow        # the name max is now bound to the function pow
>>> max(2, 5 + 5)    # operator evaluates to pow; operands to 2 and 10
1024
```

- The name `pow` is unchanged, and the function that used to be called `max` is no longer reachable by that name.
- **Multiple assignment**: all expressions on the right are evaluated *before* any name is bound. So `x, y = y, x` swaps the two values.
- `from operator import add, mul` and `from math import sqrt, pi` bind names to values from a library.

§1.3 Defining and calling functions

```
def g(y):
    x = 2 * y
    return x + 1
```

- A `def` statement creates a function and binds its name (`g`) to it in the current frame. **The body is not executed when the function is defined.** It is executed each time the function is called.
- `y` is the **formal parameter**. The first line `def g(y):` is the **signature**.
- A `return` statement evaluates its expression, ends the call, and makes that value the value of the call expression.

Procedure for calling a user-defined function.

1. Add a new **local frame**.
2. Bind the function's formal parameters to the arguments in that frame.
3. Execute the body of the function in the environment that starts with that frame.

§1.4 Frames, environments, and environment diagrams

The question. The function `g` above uses the name `x`, and the program that calls it may also use `x`. Which `x` is meant where, and can one overwrite the other?

- A **frame** holds bindings of names to values. A name appears at most once in a frame.
- The **global frame** holds the names defined at the top level (and gives access to the built-in names such as `max`, `pow`).
- An **environment** is a sequence of frames: a local frame followed by the global frame, or the global frame alone. (From Week 2 on, the sequence can be longer.)
- **Lookup rule**: a name evaluates to the value bound to it in the *earliest* frame of the current environment in which the name is found. Local frame first, then global.
- An assignment inside a function body binds the name in the *local* frame. It does not change any other frame.

The idea

Why organise names this way? The person who writes `g` should be free to choose the names `x` and `y` without knowing which names the rest of the program uses. Local frames give that: every call gets a fresh frame, the body looks there first, and assignments in the body stay there. When the call returns, the frame is no longer used.

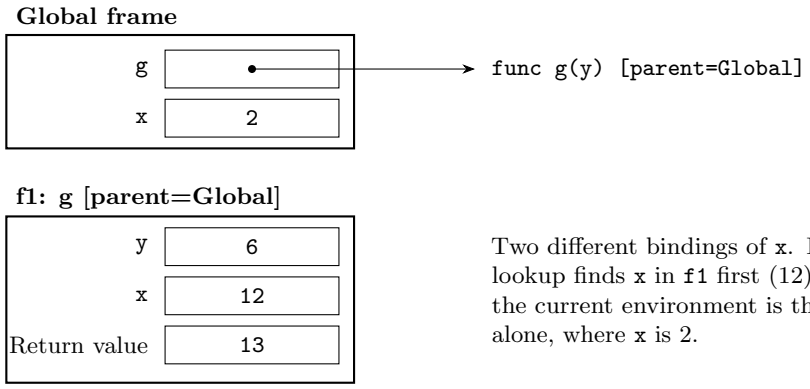
An environment diagram is the bookkeeping for these rules. Drawing one is slow but mechanical, and it removes all guessing from questions about what a name refers to.

Worked example (from lecture). Trace this program:

```
def g(y):
    x = 2 * y
    return x + 1

x = 2
print(g(3 * x) + 3)
print(x)
```

1. `def`: bind `g` in the global frame to a new function.
2. `x = 2`: bind `x` to `2` in the global frame.
3. Evaluate `g(3 * x) + 3`. First the call `g(3 * x)`: the operator `g` is the function; the operand `3 * x` is evaluated in the global frame and gives `6`.
4. Call `g` with argument `6`: new frame `f1`; bind `y` to `6`.
5. Body, line 1: `2 * y` looks up `y` in `f1`: `12`. Bind `x` to `12` *in f1*. The global `x` is still `2`.
6. Body, line 2: `x + 1` looks up `x`; it is found in `f1` first: `13`. Return `13`.
7. Back in the global frame: `13 + 3` is `16`; `print` displays `16`.
8. `print(x)`: `x` is looked up in the global frame: displays `2`.



Environment diagram at the moment `g` returns. The frame label records the function's name and its parent.

How to draw an environment diagram.

1. Start with the global frame. Execute statements one at a time, in order.
2. `def`: write the function value `func name(params) [parent=`*`current frame`*`]` and bind the name to it.
3. Assignment: evaluate the right side completely, then write or overwrite the binding in the current frame.
4. Call to a user-defined function: evaluate operator and operands first (in the current environment). Then open a new frame labelled `f1`, `f2`, ...with the function's *original* name (the name in its `def`) and its parent; bind the parameters; run the body.
5. `return`: write the return value in the frame. A function whose body finishes without `return` returns `None`.

6. Calls to built-in functions (`print`, `max`, `add`) do not get a frame in the diagram.

Name conflicts (from lecture).

```
from operator import mul
def square(square):
    return mul(square, square)
square(3)
```

- In the global frame, `square` is the function. The call `square(3)` opens `f1` and binds the *parameter* `square` to `3` there.
- In the body, `square` is looked up in `f1` first and is `3`; `mul` is not in `f1`, so it is found in the global frame. The result is `9`.
- It works, but inside the body the function can no longer be reached by the name `square`. Do not reuse names like this.

§1.5 print, return, and None

The question. `print(x)` and `return x` both seem to "give back" `x`. What is the difference, and why does `None` show up in exam answers?

- `return` hands a value to the caller. Nothing is displayed.
- `print` *displays* its arguments (separated by spaces) and *returns* `None`.
- `None` is a value that represents nothing. A function without a `return` statement returns `None`.
- A **pure function** only returns a value (`abs`, `pow`, `add`). A **non-pure function** also has a side effect, such as displaying output (`print`).
- **The interactive interpreter** (`>>>`) displays the value of each expression you type, *unless that value is `None`*. A file run with `python3 file.py` displays only what is printed.
- In the interpreter a string value is displayed with quotes (`'hi'`); `print('hi')` displays `hi` without quotes.

```
>>> -2           # value displayed
-2
>>> print(-2)    # displayed by print; the value None is not displayed
-2
>>> None         # nothing displayed
>>> print(None)  # print displays the word None
None
>>> print(print(1), print(2))
1
2
None None
```

The idea

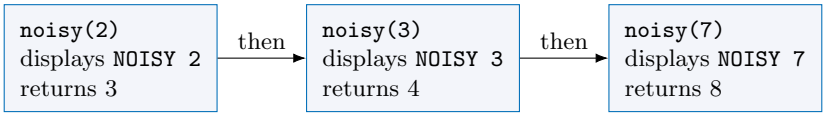
The last example is the model for every nested-print question. Apply the evaluation procedure to the outer call: its operands `print(1)` and `print(2)` must be evaluated first, left to right. Evaluating `print(1)` displays `1` and has value `None`; evaluating `print(2)` displays `2` and has value `None`. Only then is the outer `print` applied, to the arguments `None` and `None`, so it displays `None None`. The outer call's own value is `None`, which the interpreter does not display.

The lecture's `noisy` example.

```
def noisy(x):
    print('NOISY', x)
    return x + 1

>>> noisy(noisy(2) + noisy(3))
NOISY 2
NOISY 3
NOISY 7
8
```

1. The operand `noisy(2) + noisy(3)` is evaluated first. `noisy(2)` displays `NOISY 2` and returns `3`.
2. `noisy(3)` displays `NOISY 3` and returns `4`. The operand's value is `7`.
3. The outer call `noisy(7)` displays `NOISY 7` and returns `8`.
4. The interpreter displays the value `8`.



The three calls in the order they run. The outer call cannot start until its operand $3 + 4$ has a value, so it displays `NOISY 7` last. Then the interpreter displays its return value, `8`.

If `noisy` used `print(x + 1)` instead of `return x + 1`, then `noisy(2)` would return `None`, and `None + None` is an error.

Search, the lecture's closing example. Discussion 0 asked for the shortest expression built from `f(x) = x + 1`, `g(x) = 2 * x - 1`, `h(x, y)` (concatenate digits) and the number `5` that evaluates to `2026`. The lecture's program tries every expression with 3 calls, then 4 calls, and so on, and finds `f(g(h(f(g(5)),g(f(f(5))))))` with 8 calls. The point for the course: a program can examine many alternatives by repeating simple operations. The code uses classes, generators and recursion, which come later.

Shortcuts

- **What Would Python Display**: work one line at a time. For each line write, in order, (1) everything displayed while operands are evaluated, left to right and inside out, then (2) what the outermost call displays, then (3) the line's value if you are at `>>>` and it is not `None`.
- **Label every call with its return value** as you go. For `print` the label is always `None`.
- **Last digit / the rest**: `n % 10` and `n // 10`.
- **Environment diagrams**: an assignment never changes another frame; the frame label uses the name from the `def` line, not the name used in the call.
- **Is the body run now?** Not at `def`; only at a call. A name that does not exist inside a body causes an error only when that line is executed.
- **Operands before the call**: an operand is evaluated in the frame where the call expression is written, not in the new frame.

Traps

- `print` returns `None`. `x = print(5)` displays `5` and binds `x` to `None`.
- `6 / 3` is `2.0`, not `2`.
- `y = x + 1` does not link `y` to `x`.
- Rebinding a built-in name (`max = pow`) is legal and silently changes what later calls do.
- Forgetting `return`: the function returns `None`, and the error appears later, where the result is used.
- In the interpreter a string value shows quotes; printed, it does not.

§2 Problem set

These are original problems of the kind asked in lab and on Midterm 1. (The course's own homework, lab and project questions are for you to do without AI help; they are not reproduced or solved here.) Past-exam questions on these topics also use `lambda`, `and`/`or` and `while`, so they appear in the Week 2 and Week 3 handouts. The programming exercises are in `exercises/`.

P1 (evaluation order). Draw the expression tree of the call below and give its value. In which order do the five calls finish?

```
sub(pow(2, add(1, 2)), mul(2, add(1, 1)))
```

P2 (assignment). What is displayed?

```
>>> a = 3
>>> b = a * 2
>>> a = b - 1
>>> a, b = b, a + b
>>> a
>>> b
```

P3 (names bound to functions). What is displayed?

```
>>> f = min
>>> min = max
>>> max = f
>>> min(1, 2) + max(3, 4)
```

P4 (print and None). What is displayed? Write `ERROR` if an error occurs.

```
>>> x = print(1, 2)
>>> x
>>> print(x)
>>> print(print(3), 4)
>>> print(7) + 1
```

P5 (return versus print). What is displayed?

```
def loud(x):
    print('got', x)
    return 2 * x

def quiet(x):
    print(2 * x)

>>> loud(loud(1))
>>> quiet(quiet(1))
```

P6 (environment diagram). Draw the environment diagram and give what is displayed.

```
def f(x, y):
    x = x + y
    y = x * 2
    return y - x

x = 3
y = f(x, x + 1)
print(x, y)
```

P7 (parameters shadow globals). What is displayed?

```
from operator import add
def add(add, mul):
    return mul

x = add(1, 2)
print(x)
print(add(add, x))
```

P8 (body runs only when called). Which line, if any, causes an error, and when?

```
def h(n):
    return n + undefined_name

z = 10
print(z)
print(h(z))
```